Аутентификация через Keycloak

В системе два режима аутентификации, переключаемых переменной AUTH_MODE:

Режим	Что проверяет	Когда используется
jwt (по умолчанию)	HS256-токен, подписанный SECRET_KEY	Локальный запуск, демонстрация, оба облачных контура
keycloak	RS256-токен Keycloak по JWKS	Закрытый контур заказчика, корпоративный SSO

Оба режима возвращают локальную запись User: она нужна для связей в БД (автор комментария, ответственный KAM) и для RBAC. Различается только способ подтверждения личности.

Почему JWT остаётся режимом по умолчанию

В ТЗ Keycloak указан как вариант авторизации. Для прототипа JWT не требует отдельного сервиса, поэтому демонстрационный контур работает без Keycloak. Путь к корпоративному SSO при этом реализован полностью и включается одной переменной — переписывать код при требовании заказчика не нужно.

Состав

Файл	Назначение
keycloak/realm-export.json	Realm rtk-crm, клиент, роли, демо-пользователи
backend/auth/keycloak.py	Проверка токена по JWKS, разбор ролей
backend/auth/security.py	Выбор режима, RBAC по ролям токена
tests/test_keycloak_auth.py	20 тестов: подпись, claims, RBAC, кэш ключей

Структура realm

Realm rtk-crm содержит три роли и публичный клиент с PKCE:

- admin — администратор: справочники, пользователи, настройки;
- manager — менеджер: справочники и отчёты по всем вузам;
- kam — KAM: заявки и отчёты по своим вузам.

Клиент rtk-crm-frontend — публичный (без client secret), Authorization Code Flow с pkce.code.challenge.method=S256. Для публичного клиента PKCE обязателен: без него перехваченный код авторизации можно обменять на токен.

Роли кладутся в токен двумя мапперами: realm-роли (realm_access.roles) и audience (aud). Второй нужен, потому что по умолчанию Keycloak не проставляет aud клиента, а без него проверка audience не проходит.

Адреса возврата заданы шаблонами: http://localhost:3000/*, http://localhost:5173/* для локального запуска и http://* для стенда. Последний шаблон нужен, потому что адрес ВМ заранее неизвестен, а Keycloak принимает * только в конце строки: запись вида http://*/* не совпадает ни с одним адресом, и вход падает с Invalid parameter: redirect_uri.

Как работает проверка токена

```
Запрос с Bearer-токеном
        │
        ├─ AUTH_MODE=jwt ──────► HS256 по SECRET_KEY, sub = id пользователя
        │
        └─ AUTH_MODE=keycloak ─► RS256 по публичному ключу realm
                                 │
                                 ├─ kid из заголовка токена
                                 ├─ ключ из кэша JWKS (TTL 300 с)
                                 ├─ проверка iss / aud / exp
                                 └─ email или preferred_username
                                        │
                                        ├─ запись найдена → вход
                                        └─ записи нет → создаётся автоматически
```

Особенности реализации:

- **Кэш ключей.** Запрос к Keycloak делается не на каждый HTTP-запрос, а раз в `KEYCLOAK_JWKS_TTL_SECONDS`. Если в токене встретился неизвестный `kid`, кэш сбрасывается и ключи запрашиваются заново — так обрабатывается ротация.
- **Сбой Keycloak — это 503, а не 401.** Если JWKS недоступен, ответ `503 Service Unavailable`: токен может быть корректным, и клиент не должен разлогиниваться из-за сетевой ошибки.
- **Роли из токена — источник истины.** В режиме keycloak RBAC опирается на `realm_access.roles` и `resource_access.<client>.roles`, а не на роль в БД. Смена роли в Keycloak действует сразу, без правки учётки в CRM.
- **Автосоздание учётки.** Если пользователь есть в Keycloak, но ещё не в CRM, запись создаётся при первом входе с ролью по приоритету admin > manager > kam. Локальный вход паролем для таких учёток невозможен: в `hashed_password` записан маркер `!keycloak`, который не является корректным bcrypt-хешем.

Запуск локально

```
export KEYCLOAK_ADMIN_PASSWORD=<пароль-админа-realm>
export DATABASE_URL=sqlite+aiosqlite:///./dev.db
export MINIO_ROOT_PASSWORD=<пароль-minio>
docker compose up -d keycloak

# Проверка, что realm импортирован
curl -s http://localhost:8080/realms/rtk-crm/.well-known/openid-configuration | head
```

Realm импортируется из `keycloak/realm-export.json` при первом старте контейнера (`--import-realm`). Консоль администратора — http://localhost:8080.

Демо-пользователи из realm-файла:

Логин	Пароль	Роль
`admin@rtk.ru`	`Admin123!`	admin
`manager@rtk.ru`	`Manager123!`	manager
`kam@rtk.ru`	`Kam123!`	kam

Пароли в realm-файле предназначены для локальной демонстрации. Для закрытого контура их нужно заменить при импорте realm.

Включение в приложении

```
AUTH_MODE=keycloak
# Внутренний адрес: по нему backend забирает публичные ключи (JWKS).
KEYCLOAK_URL=https://keycloak.example.ru
# Публичный адрес: по нему браузер открывает Keycloak. Именно он попадает в
# claim `iss`, поэтому по нему проверяется issuer токена. В Docker это
# http://keycloak:8080 для KEYCLOAK_URL и внешний адрес ВМ для этого параметра.
KEYCLOAK_PUBLIC_URL=https://keycloak.example.ru
KEYCLOAK_REALM=rtk-crm
KEYCLOAK_CLIENT_ID=rtk-crm-frontend
# audience токена; пустое значение отключает проверку aud
KEYCLOAK_AUDIENCE=rtk-crm-frontend
KEYCLOAK_JWKS_TTL_SECONDS=300
```

Фронтенд переключается тем же режимом при сборке: `VITE_AUTH_MODE=keycloak` (в Docker Compose значение берётся из `AUTH_MODE`). В этом режиме вход выполняет Keycloak по Authorization Code Flow с PKCE, а access token подставляется в запросы и продлевается перед их отправкой. При `VITE_AUTH_MODE=jwt` остаётся локальная форма входа по email и паролю.

Адрес Keycloak для браузера задаётся через `KEYCLOAK_PUBLIC_URL`: из него собирается `VITE_KEYCLOAK_URL`, поэтому на ВМ достаточно указать `KEYCLOAK_PUBLIC_URL=http://<ip-вм>:8080`, и вход заработает без правки образа.

Страница входа

В режиме `keycloak` на странице входа доступны обе опции: кнопка **Войти через Keycloak** и форма email/пароль под разделителем. Первая ведёт в единую систему входа, вторая остаётся рабочим запасным вариантом.

Перед инициализацией keycloak-js фронтенд проверяет доступность Keycloak по `.well-known/openid-configuration`. Это нужно потому, что при недоступном сервере keycloak-js в режиме `check-sso` уводит верхнее окно на Keycloak и оставляет пользователя на странице ошибки браузера. Предпроверка не даёт этому произойти: если сервер не ответил, страница сразу показывает форму входа.

Получение токена

Парольный грант (удобно для отладки API; в проде клиент использует Authorization Code + PKCE):

```
curl -s -X POST \
  http://localhost:8080/realms/rtk-crm/protocol/openid-connect/token \
  -d grant_type=password \
  -d client_id=rtk-crm-frontend \
  -d username=admin@rtk.ru \
  -d password='Admin123!' | python3 -m json.tool
```

Использование токена:

```
TOKEN=$(curl -s -X POST \
  http://localhost:8080/realms/rtk-crm/protocol/openid-connect/token \
  -d grant_type=password -d client_id=rtk-crm-frontend \
  -d username=admin@rtk.ru -d password='Admin123!' \
  | python3 -c 'import sys,json;print(json.load(sys.stdin)["access_token"])')

curl -s http://localhost:8000/api/auth/me -H "Authorization: Bearer $TOKEN"
```

Переход с JWT на Keycloak

1. Поднять Keycloak, импортировать realm, проверить JWKS.
2. Убедиться, что email пользователей CRM совпадает с email в Keycloak: сопоставление учётных записей идёт по этому полю.
3. Выставить `AUTH_MODE=keycloak` и перезапустить backend.
4. Проверить вход: `GET /api/auth/me` с токеном Keycloak.

Обратный переход — вернуть `AUTH_MODE=jwt`. Оба режима сосуществуют, данные пользователей не меняются.

Ограничения текущей реализации

- **Хранение состояния.** Локальный запуск использует `start-dev` с встроенной БД H2. Для закрытого контура нужен `start --optimized` с внешним PostgreSQL и включённым HTTPS: в dev-режиме Keycloak не рассчитан на нагрузку.
- **`sslRequired: none`** в realm-файле допускает работу по HTTP: демо-стенд открывается по `http://<ip-вм>:8080`, и при `external` Keycloak отклонял бы такой вход. Для промышленного контура требуется вернуть `external` и включить TLS на всех участках.
- **Выход из системы.** Logout выполняется на стороне Keycloak; backend не ведёт список отозванных токенов, поэтому access-токен живёт до `exp`. Уменьшение `accessTokenLifespan` в realm сокращает это окно.